

Case Study: Re-architecting a Technician Resource Portal

Stack: Go, AWS Lambda, AWS EventBridge, AWS S3, GitHub Actions, Google Drive API, Terraform, Prometheus, Grafana

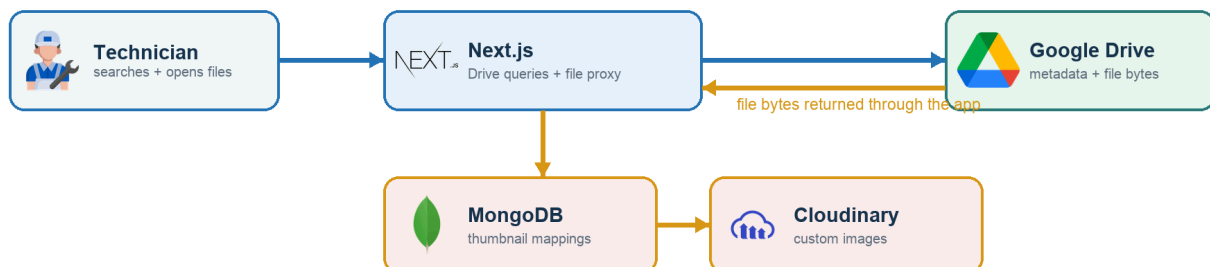
The Challenge: Scaling Document Access

otasupport.com publishes thousands of Google Drive files for technicians. Originally, the shared Drive experience was **difficult to search**. Relying on manual permissions was an operational bottleneck that scaled poorly as the team and library grew.

To solve this, the first custom website was built. While it successfully provided a dedicated portal for technicians, the underlying architecture struggled to scale efficiently with user demand and eventually grew beyond the initial problem.

The Initial Architecture: Next.js & Live Proxy

<https://v1.otasupport.com>



The initial application **fetches data from Google Drive dynamically on every refresh**. It proxied file downloads through Vercel and maintained thumbnail mappings using MongoDB and Cloudinary.

As traffic grew to 4,573 visitors and 13,200 page views over a 12-month period, this architecture revealed several critical flaws:

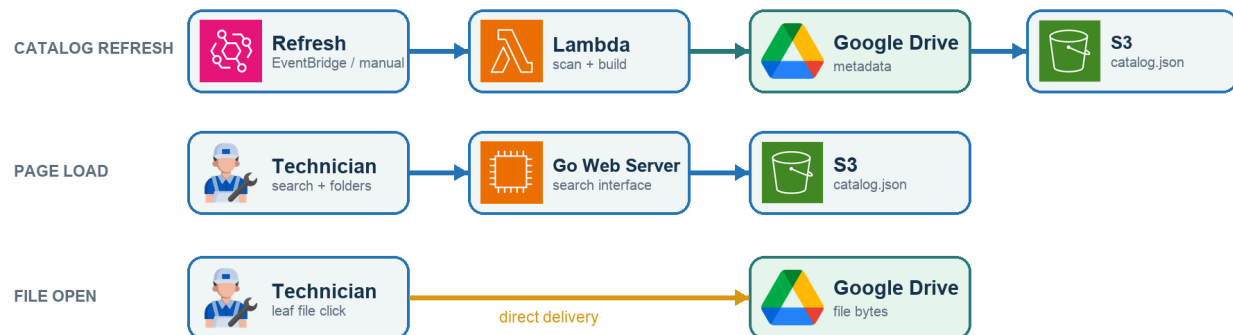
- **Unsustainable Bandwidth:** Proxying files added significant application bandwidth and runtime pressure. Over 12 months, this generated **639 GB¹** of outbound transfer and over 1 million edge requests.

¹ https://drive.google.com/file/d/1XMOk7Zc_9tosxqHrZ3B639i1dLHm8GhS/view?usp=drive_link

- **Delivery Failures:** Because the application sat directly in the delivery path, **large PDF files would frequently fail to load.**
- **Performance Bottlenecks:** Because the application sat as a live middleware, it queried Google Drive's API on the fly for every user request. This was inherently slow and resource-heavy, causing significant lag before technicians could even begin searching for files.
- **Complex Content Management:** Managers had to follow **complicated content rules²**. For example, they had to wrap text in parentheses to prevent it from displaying on the UI, and keep unpublished files strictly in the Root Folder.

The Newer Architecture: Precompute the Catalog

<https://otasupport.com>



Recognizing that the core requirement was simply to index the library rather than relay every file, the system was re-architected using Go and AWS. The guiding principle for V2 was to offload all catalog generation to a background service, eliminating the need to query Google Drive on the fly

- **Decoupled Catalog Generation:** Catalog generation was separated from customer traffic. An AWS Lambda function, triggered by EventBridge once a day (or via a manual refresh), scans Drive and updates the metadata catalog in private S3.
- **Direct Delivery:** A Go application running on EC2 serves a simple web interface. Search runs directly in the browser, and leaf files open directly from Google Drive. This completely bypasses the application's bandwidth.

² https://drive.google.com/file/d/1NRU3g72RnCBInRvOTcQ3ZR2cRFcW-fek/view?usp=drive_link

- **Simplified Management:** Google Drive is kept as the source of truth, allowing managers to continue using familiar folders without copying any customer files to AWS.

Impact & System Comparison

By redesigning the application around a precomputed catalog rather than live proxying, the system achieved drastic improvements in speed, bandwidth efficiency, and architectural simplicity.

Area	V1 (Next.js Middleware)	V2 (Go Search-First Catalog)
Customer Request	Live Drive work	Prebuilt S3 catalog
File Delivery	Proxied by application	Direct from Google Drive
Supporting Systems	Next.js + MongoDB + Cloudinary	Go service + one JSON snapshot
Catalog Response	About 3 seconds	About 0.32 seconds
Outbound Transfer	639.91 GB / 12 months	About 1.65GB ³

V1 treated the application server as a pass-through proxy for Google Drive binary data. V2 re-framed the server as an indexer—storing a tiny, static JSON snapshot in S3—and delegated file delivery directly to Google Drive. This eliminated massive application-level bandwidth costs, completely resolved large PDF download timeouts, and reduced catalog response times by nearly 10x.

Observability & Monitoring⁴

To gain insight into system health and technician usage, observability was added using Grafana, Prometheus, and CloudWatch. Rather than introducing the complexity of Kubernetes, the monitoring tools run via Docker Compose alongside the Go web server on the existing EC2 host. The system tracks core engagement events—such as page views, folder navigation, and file opens—while strictly excluding sensitive data like search queries or authorization headers.

³ 13,200 (annual page views) * 121KB (catalog file size) ≈ 1.6GB

⁴ <http://otasupport.com/admin/monitoring>

Capstone Project: Plant Disease Detection

Role: AI Engineer

Core Technologies: Computer Vision, Multilabel-Classification, CycleGAN

Project Overview¹

Our team developed a plant pathology diagnosis application to detect and classify Korean crop diseases from a leaf image. The final production model successfully classifies 72 distinct classes, covering 50 different diseases and normal states across 22 types of agricultural crops.

Data Curation & Final Performance

Our team scaled the initially provided dataset (3 crops, 8 diseases) into a massive, robust dataset by sourcing additional data from AI HUB, Kaggle, and academic researchers.

- **Total Dataset:** Curated a final dataset consisting of 33,091 training images and 11,044 evaluation images.
- **Metric Selection (Macro-F1):** Due to the significant class imbalance across the 72 categories (e.g., 2,268 training samples for Red Bean Powdery Mildew vs. only 113 for Sesame Powdery Mildew), the **Macro F1-score** was chosen as the primary evaluation metric. By calculating the metric independently for each class and averaging them equally, this ensures the model's performance isn't skewed by majority classes and reliably detects even the rarest diseases.
- **Performance:** Achieved an overall macro average F1-score of 0.971, with near-perfect (1.00) F1-scores on critical crops like Apple, Eggplant, Strawberry, Tomato, and Watermelon.

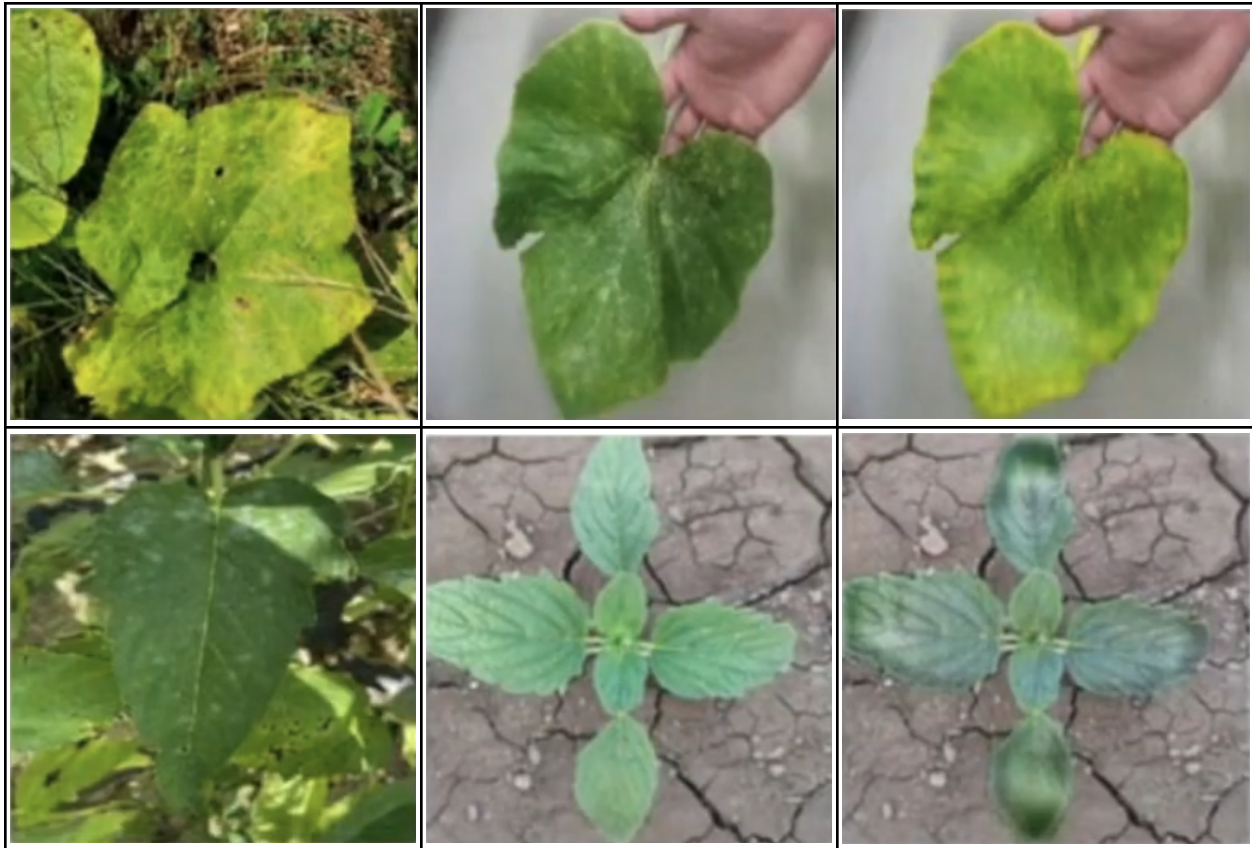
Key AI Methodologies & Contributions

CycleGAN² for Synthetic Data Generation

Disease Data	Normal Data	Generated Disease
--------------	-------------	-------------------

¹ <https://github.com/JeongjunBae/debug-AI/blob/main/README-EN.md>

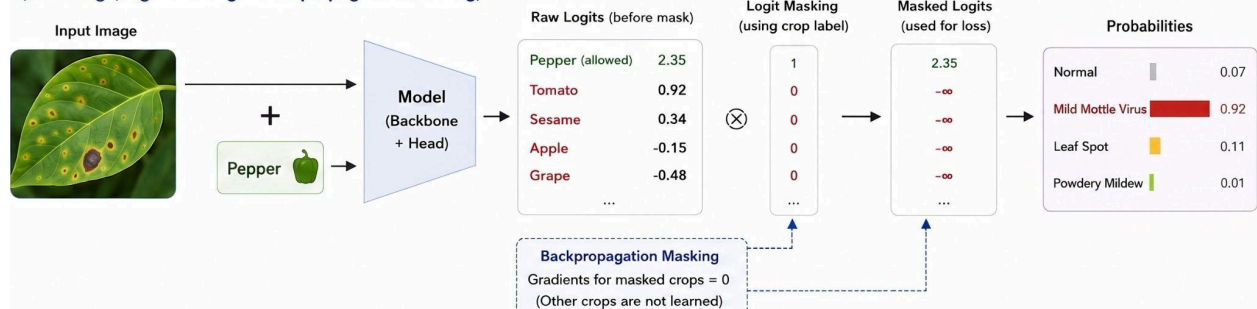
² <https://arxiv.org/abs/1703.10593>



Addressed the high cost and logistical difficulty of acquiring diseased plant images (which typically require isolated environments and expert virus injection). CycleGAN generates realistic diseased images from cheaper, more accessible normal images

Crop-aware Architecture

1) Training (Logit Masking + Backpropagation Masking)



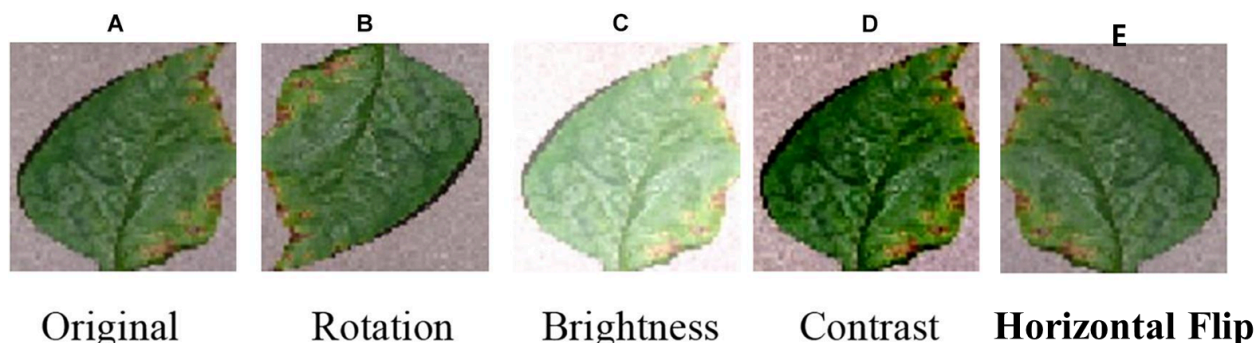
2) Inference (Logit Masking)



Capitalized on the premise that identifying the base crop type is trivial compared to diagnosing the specific disease. Engineered a contextual filtering mechanism that feeds the crop type alongside the image during both training and inference.

- **Training:** During training, backpropagation for non-target crop types is zeroed out.
- **Inference:** During inference, the model masks the confidence scores of irrelevant crop types (setting them to 0), drastically reducing inter-class confusion.

Robust Data Augmentation



Designed a comprehensive augmentation pipeline to further improve model generalization on field data.

Ablation Study & Optimization³

Conducted a rigorous ablation study to validate the architectural choices. The baseline model achieved a 0.8729 Validation Macro F1-score. Integrating PlantGAN improved performance to 0.8971, while adding the Crop-aware architecture drove the score up to 0.9655. The final integration of the full augmentation pipeline resulted in the state-of-the-art 0.9710 Validation Macro F1-score.

Ablation Study Table

GAN	Crop-aware	Augmentation	Validation Macro F1-Score
X	X	X	0.8729
O	X	X	0.8971
O	O	X	0.9655
O	O	O	0.9710

Awards

Third Prize⁴: 2022 Sejong University Creative Comprehensive Design Competition.

Popularity Prize⁵: 2022 Sejong University Creative Comprehensive Design Competition.

³ https://drive.google.com/file/d/1snUUTO9Awm21kvphDfDeMOblA1Oxu7kQ/view?usp=drive_link

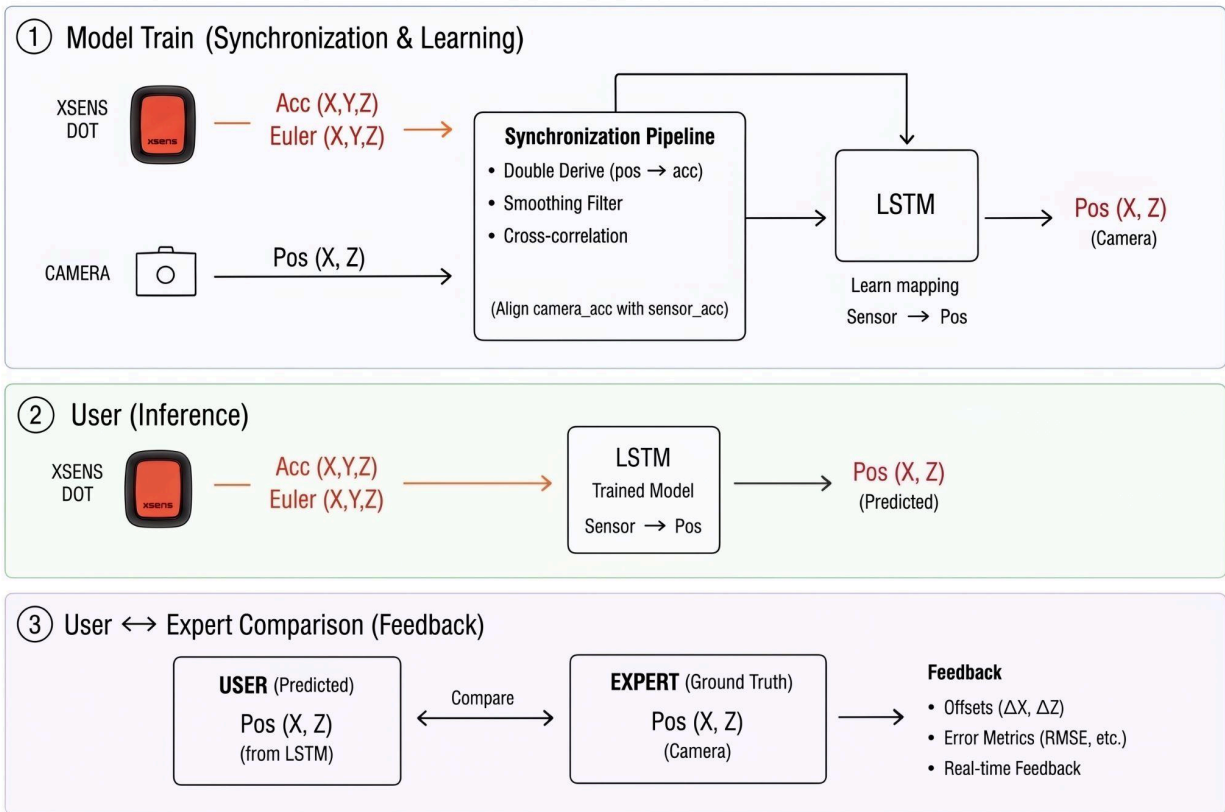
⁴ https://drive.google.com/file/d/1bahM5ottudMM88nuXQMMx_oCmH7-B7SA/view?usp=drive_link

⁵ https://drive.google.com/file/d/1aOuhs1cmdkzZmD5gbf9-kHjqTDATsRp/view?usp=drive_link

Project Summary: Exercise Correction Using Wearable Devices

Role: Machine Learning Engineer

Project Overview¹



We developed a sophisticated software and hardware ecosystem to provide real-time exercise correction. The project operates on a three-step machine learning architecture:

- 1. Model Train (Synchronization & Learning):** The LSTM model is trained using synchronized data from both an XSENS DOT IMU wearable sensor (acceleration and Euler angles) and a camera (positional data X, Z). The synchronization pipeline aligns the data by double-deriving the camera's position to acceleration, applying a smoothing filter, and using cross-correlation. The camera's positional data serves as the definitive answer labels for the LSTM to learn the sensor-to-position mapping.
- 2. User (Inference):** Once the mapping is learned, users perform exercises using only the XSENS DOT wearable sensor. The trained LSTM model takes the real-time sensor data and outputs the predicted position (X, Z) without requiring any camera input.
- 3. User ↔ Expert Comparison (Feedback):** The user's predicted positional data is compared directly against an expert's ground truth positional data. By

¹ <https://github.com/JeongjunBae/Exercise-correction-using-wearable-devices/blob/main/README-EN.md>

calculating positional offsets (ΔX , ΔZ) and error metrics (such as RMSE), the system provides actionable, real-time posture feedback to the user.

For a visual and detailed reference regarding the project's background and development, please refer to the presentation slides². Please note that a band-pass filter may be used for some exercises.

Performance Metrics:

Through this pipeline, the system achieved high precision in tracking, resulting in:

- Vertical Error: **2.5cm**
- Horizontal Error: **4cm**

Background & Motivation

- Although wearable-device-based motion tracking has been actively studied, **existing approaches often require numerous sensors**, making them impractical for everyday users to wear during exercise.
- The **high cost of personal training** creates a significant barrier to accessible and proper posture correction.
- Muscle imbalances are highly common. Individuals naturally default to unbalanced and potentially harmful postures when exhausted during intense weight training.

Technical Implementation Overview

Component	Technology / Approach	Details
IMU Sensors	Xsens Dot	Utilized a highly accurate MEMS sensor for raw data gathering.
Vertical Tracking	Band-pass & High-pass filters	Mitigated integration drift by isolating the vertical position through sequential filtering.
Horizontal Tracking	LSTM Neural Network	Mapped horizontal acceleration directly to camera tracking data to avoid exponential drift.

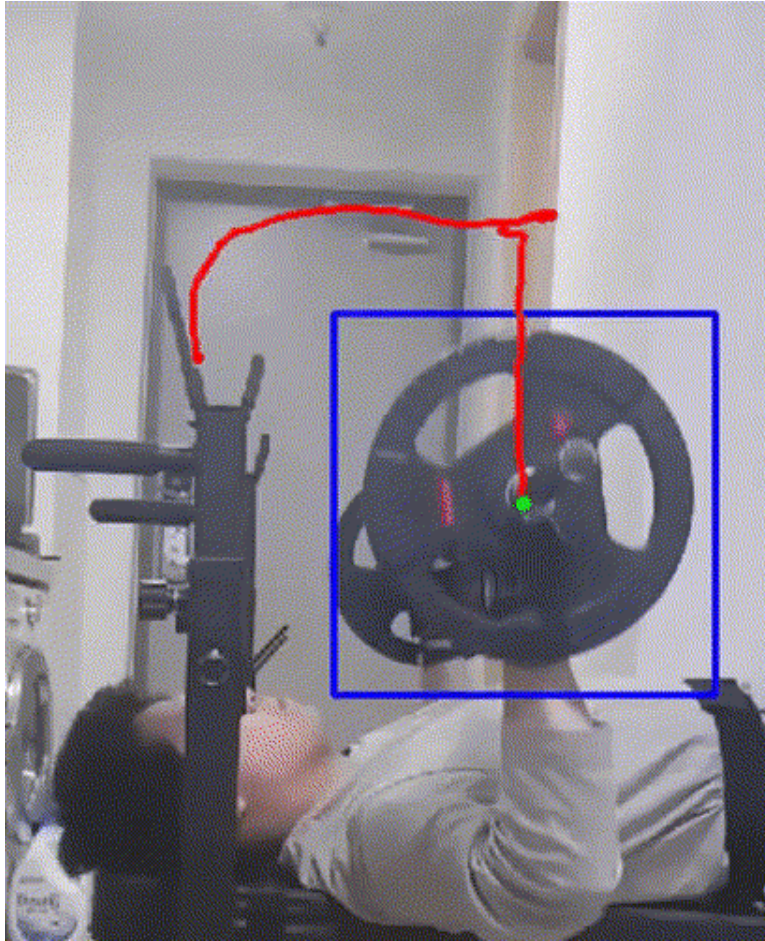
Algorithms & Data Processing

- **Missing Data Handling:** Missing data points caused by sensor transmission drops were handled by utilizing intersecting timeframes and applying linear interpolation.
- **Synchronization³:** Data synchronization between the IMU sensor and the camera was achieved using Fourier Transforms.

² https://github.com/JeongjunBae/Exercise-correction-using-wearable-devices/blob/main/final_presentation_English.pdf

³ https://github.com/JeongjunBae/Synchronize_Signals/blob/main/ReadMe-EN.md

- **Computer Vision:** The pipeline sets a physical scale based on the barbell's size and precisely tracks the center of the weight plates.



- **Exercise Evaluation:** The algorithm detects signal peaks to compare maximum heights and reach times. It further evaluates the exercise cycle by measuring the time between peaks and calculates standard deviations to ensure cycle consistency.

Challenges

- Mapping acceleration directly to position yielded less-than-ideal results initially, likely due to the differing physical properties of the metrics.
- The third-party WL-Analysis app⁴ failed to recognize several exercises, forcing us to quickly develop our own custom tracking program.

Future Work

- Design more robust algorithms to further minimize horizontal positional errors.
- Increase the system's capabilities beyond the currently supported exercises (Military Press, Squat, Barbell Row, and Deadlift).

Awards

- **First Prize:** 2021 Kyung Hee University Creative Design Competition.⁵
- **Second Prize:** 2021 Sungkyunkwan University (SKKU) Creative Design Competition.⁶

⁴ https://play.google.com/store/apps/details?id=com.karolsmolak.wlanalysis&hl=en_US&pli=1

⁵ https://drive.google.com/file/d/11RK3T1I0vtgmYwRjneoGL2eivUWNryM/view?usp=drive_link

⁶ https://drive.google.com/file/d/1ITSIbwsLu7ajZHV0UYx7xbmm7YR7AM7f/view?usp=drive_link